

Perdiendo el miedo a la programación concurrente

Hoja de referencia condensada a partir de la charla de Manuel Montenegro (Facultad de Informática) — sintaxis, procesos, nodos y comunicación distribuida.

Contenido original: Manuel Montenegro · Edición, verificación y actualización:

MC. René Solís R.

BEAM / Erlang · Ericsson, 1986 **Paradigma** funcional + concurrente, dinámicamente tipado

Filosofía "let it crash"

Verificado con Erlang/OTP 29.0.5 (ago. 2026) — todos los ejemplos compilados y ejecutados

§1 Fundamentos del lenguaje

Erlang nace en Ericsson en 1986 para telefonía, donde perder una llamada no es una opción. Ese origen explica cada decisión de diseño.

PRINCIPIOS DE DISEÑO

- **Todo es un proceso.** No hay "hilos" ligeros por un lado y objetos por otro: la unidad de concurrencia es el proceso Erlang.
- **Crear y destruir procesos debe ser ligero y eficiente** — se pueden lanzar cientos de miles sin problema.
- **Los procesos no comparten memoria.** Cada uno vive aislado.
- **La única comunicación es paso de mensajes** — nunca variables compartidas ni locks.
- **Filosofía "Let it crash".** No se defiende cada función con try/catch: se deja morir el proceso y otro lo supervisa y reinicia.

§2 Las seis rarezas

Lo que más sorprende a quien llega desde un lenguaje imperativo, en el orden en que aparecen al aprender Erlang.

01 Hay que escribir un **punto (.)** en la shell para finalizar una expresión.

02 **Todas las variables son inmutables.** Una vez ligada, no se le puede asignar otro valor distinto (usa `f(X)` o `f()` en la shell para "olvidarla").

03 **En Erlang no hay bucles.** La iteración se logra con recursión.

04 El mecanismo central del lenguaje es **el ajuste de patrones** (pattern matching): así se accede a tuplas, se elige cláusula de función y se desestructuran mensajes.

05 **Los caracteres se representan mediante números enteros.** `$A` es literalmente `65`.

06 **Las cadenas de texto son listas de caracteres** — heredan todas las funciones y operadores de las listas.

§3 Sintaxis esencial

SHELL, ARITMÉTICA Y ÁTOMOS

```
1> 1 + 5.  
6  
2> 3 * 9 - 4.  
23  
3> hola.      átomo: constante simbólica, minúscula inicial  
hola  
4> 5 =< 4.    true/false son átomos  
false
```

VARIABLES Y ÁMBITO EN LA SHELL

```
1> X = 4.      mayúscula inicial
4
2> X = 5.      ¡error! X ya está ligada a 4
** exception error: no match of right hand side value 5
3> f(X).      olvida solo X
ok
4> f().      olvida todas las variables
ok
```

MÓDULOS Y FUNCIONES

```
%% hola_mundo.erl
-module(hola_mundo).
-export([saludar/0, saludar/1]).

saludar() ->
  io:format("Hola, mundo!~n").

saludar(Nombre) ->
  io:format("Hola, ~s~n", [Nombre]).
```

```
1> c(hola_mundo).
{ok, hola_mundo}
2> hola_mundo:saludar("Gloria").
Hola, Gloria!
```

La aridad forma parte del nombre: `saludar/0` y `saludar/1` son funciones distintas que pueden coexistir.

FORMATO DE `IO:FORMAT`

MARCADOR	SIGNIFICADO
<code>~n</code>	fin de línea
<code>~s</code>	cadena de texto
<code>~w</code>	término Erlang arbitrario (write)

TUPLAS Y AJUSTE DE PATRONES

```
decrement(0) -> {error, already_zero};
decrement(Num) -> {ok, Num - 1}.
```

```
1> {ok, Z} = tuple_examples:decrement(5).
{ok, 4}
2> {ok, V} = tuple_examples:decrement(0).    el patrón {ok, _} no ajusta con {error, ...}
** exception error: no match of right hand side value {error,already_zero}
```

CLÁUSULA **CASE ... OF ... END**

```
decrement_print(N) ->
case decrement(N) of
{ok, NDec} ->
    io:format("Resultado: ~w~n", [NDec]);
{error, _} ->
    io:format("Error al decrementar.~n")
end.
```

§4 Listas y cadenas

```
1> Xs = [1, 3, ok, 10, 4, "Hola"].    [ ] = lista vacía
[1,3,ok,10,4,"Hola"]
2> Xs ++ [10, 30].                    ++ concatena listas
[1,3,ok,10,4,"Hola",10,30]
3> lists:reverse(Xs).
["Hola",4,10,ok,3,1]
4> lists:member(3, Xs).
true
```

LISTAS INTENSIONALES (LIST COMPREHENSIONS)

```

1> [ 2*X + 1 || X <- [1,2,3,4,5] ].
[3,5,7,9,11]
2> [ 3*X || X <- [1,2,3,4,5], X rem 2 == 0 ]. filtro tras la coma
[6,12]
3> [ {X, Y} || X <- [1,2,3], Y <- [a,b] ]. generadores anidados
[{1,a},{1,b},{2,a},{2,b},{3,a},{3,b}]

```

CARACTERES Y CADENAS (RAREZAS 5 Y 6)

```

1> $A.
65
2> Cadena = "Qué hay de nuevo?".
"Qué hay de nuevo?"
3> [ C || C <- Cadena, not lists:member(C, "aeiou") ]. filtrar vocales
"Qé hy d nv?"

```

Ejemplo completo: sustituir letras A–Z por , salvo las incluidas en .

```

hide_char(X, Chars) when X >= $A, X <= $Z ->
  case lists:member(X, Chars) of
    true -> X;
    false -> $_
  end;
hide_char(X, _) -> X.

hide_string(String, Chars) ->
  [ hide_char(X, Chars) || X <- String ].

```

```

1> string_utils:hide_string("HOLA, ME LLAMO MANU", "ALM").
"__LA, M_ LLAM_ MA__"

```

§5 Funciones y recursión

Sin bucles (rareza 3), toda iteración se expresa como caso base + caso recursivo, opcionalmente restringido con `when`.

```
saludar(0, _) -> ok;                %% caso base
saludar(Contador, Nombre) when Contador > 0 -> %% caso recursivo
    io:format("Hola, ~s (~w)~n", [Nombre, Contador]),
    saludar(Contador - 1, Nombre).
```

Para un contador *ascendente* se añade un parámetro extra que guarda el límite, y una cláusula de "llamada inicial" con menos aridez:

```
saludar_asc(Contador, Max, _) when Contador > Max -> ok;
saludar_asc(Contador, Max, Nombre) ->
    io:format("Hola, ~s (~w)~n", [Nombre, Contador]),
    saludar_asc(Contador + 1, Max, Nombre).

saludar_asc(Max, Nombre) -> saludar_asc(1, Max, Nombre). %% llamada inicial
```

Este mismo patrón — un parámetro que actúa como *acumulador de estado* entre llamadas recursivas — es la base de los procesos con estado en [§6](#).

§6 Procesos: spawn, receive, !

SPAWN/3 — CREAR UN PROCESO

```
1> Pid = spawn(concurrente, saludar, [4, "Gloria"]).
Hola, Gloria (4)
<0.146.0>
```

`spawn(Modulo, Funcion, [Args])` ejecuta la función en un **proceso nuevo**, concurrente a quien lo invocó, y devuelve su **PID** (Process Identifier) — la referencia que se usa para enviarle mensajes, matarlo o nombrarlo.

ENVIAR Y RECIBIR

Enviar con el operador `!`:

```
Pid ! Mensaje
```

Recibir bloquea hasta que llegue algo:

```
recibir() ->  
receive  
  Msg -> io:format("~w~n",[Msg])  
end.
```

Cuando la función termina, **el proceso muere**. Para seguir escuchando mensajes hace falta un **bucle de mensajes**: una llamada recursiva final dentro de la propia cláusula.

BUCLE DE MENSAJES + ESTADO

```
contador(N) ->  
  receive  
  imprimir ->  
    io:format("Valor actual del contador: ~w~n", [N]),  
    contador(N);  
  incrementar ->  
    contador(N + 1)  
end.
```

El parámetro `N` hace de **estado**: cada mensaje procesado da lugar a una nueva llamada recursiva con el valor actualizado — no hay variables mutables, pero el proceso "recuerda".

```
1> PC = spawn(concurrente, contador, [0]).  
2> PC ! incrementar.  
3> PC ! incrementar.  
4> PC ! imprimir.  
Valor actual del contador: 2
```

VARIOS PATRONES EN UN `RECEIVE`

```

aritmetica() ->
  receive
  {suma, X, Y} ->
    io:format("~w + ~w = ~w~n", [X, Y, X + Y]),
    aritmetica();
  {resta, X, Y} ->
    io:format("~w - ~w = ~w~n", [X, Y, X - Y]),
    aritmetica()
end.

```

REGISTRAR UN PROCESO CON NOMBRE

```

1> register(mi_contador, PC).    asigna el nombre mi_contador al Pid PC
true
2> mi_contador ! imprimir.      ya no hace falta el Pid

```

§7 Comunicación bidireccional

El operador `!` es de un solo sentido. Para que un servidor responda, necesita el PID de quien preguntó — y ese PID viaja *dentro* del mensaje.

```

aritmetica_respuesta() ->
  receive
  {From, {suma, X, Y}} ->
    From ! {resultado, X + Y},
    aritmetica_respuesta()
end.

```

Quien envía se identifica con `self()`, que devuelve el PID del proceso que la invoca:

```
1> PA = spawn(concurrente, aritmetica_respuesta, []).
2> PA ! {self(), {suma, 3, 5}}.
3> flush().    imprime los mensajes que ha recibido la shell
Shell got {resultado,8}
ok
```

Este patrón `{From, Peticion}` → `From ! Respuesta` es la base de todo servidor request/response en Erlang (y, ya con supervisión, de `gen_server` en OTP).

§8 Nodos

Un **nodo** es un sistema Erlang en ejecución, identificado por un nombre (`nodo1@PT0520`). Un mismo nodo aloja muchos procesos.

```
{ NombreProceso, NombreNodo } ! Mensaje
```

Para alcanzar un proceso en otro nodo se usa el mismo operador `!`, pero el destino es un par (*nombre registrado, nodo*) — el proceso remoto **debe estar registrado con un nombre**, ya que su PID no es visible desde fuera.

```
— nodo1@PT0520 —
1> P = gtm_server:start("Bohemian Rhapsody").
2> register(gtm, P).
true
```

```
— nodo2@PT0520 —
1> {gtm, nodo1@PT0520} ! {self(), {guess, $A}}.
2> flush().
Shell got {hit,2,"____A__A____"}
```

§9 Entre máquinas

El mismo mecanismo de nodos funciona sin cambios entre máquinas distintas conectadas en red: la distribución es transparente al código.

ACCIÓN	MENSAJE
Pedir el título oculto	<code>{gtm, nodoclient@PT05XX} ! {self(), get_title}</code>
Proponer una letra	<code>{gtm, nodoclient@PT05XX} ! {self(), {guess, \$X}}</code>
Leer la respuesta	<code>flush().</code>

Nada distingue, a nivel de código, enviar un mensaje a un proceso local, a un proceso en otro nodo de la misma máquina o a uno en otra máquina de la red: siempre `Destino ! Mensaje`.

§10 Referencia rápida

OPERADORES Y SINTAXIS

SÍMBOLO	USO
<code>.</code>	termina una expresión en la shell
<code>%</code>	comentario de línea
<code>=</code>	ajuste de patrones (no asignación destructiva)
<code>++</code>	concatenación de listas
<code>!</code>	envío de mensaje: <code>Pid ! Mensaje</code>
<code> </code>	separador en listas intensionales
<code>\$c</code>	código entero del carácter <code>c</code>
<code>_</code>	patrón comodín (no liga variable)
<code>when</code>	guarda sobre una cláusula de función/receive

FUNCIONES MÁS USADAS

FUNCIÓN	HACE
<code>c(Módulo)</code>	compila <code>Módulo.erl</code> en la shell
<code>f(X)</code> / <code>f()</code>	olvida el valor de <code>X</code> / de todas las variables (solo shell)
<code>spawn(M, F, Args)</code>	crea un proceso, devuelve su PID
<code>self()</code>	PID del proceso actual
<code>register(Nombre, Pid)</code>	asocia un nombre a un PID
<code>flush()</code>	imprime los mensajes recibidos por la shell
<code>io:format(Fmt, Args)</code>	imprime con marcadores <code>~n ~s ~w</code>
<code>length(Lista)</code>	número de elementos
<code>lists:reverse(L)</code>	invierte una lista
<code>lists:member(X, L)</code>	¿pertenece <code>X</code> a <code>L</code> ?

DESTINOS DE MENSAJE SEGÚN ALCANCE

ALCANCE	FORMA
Mismo nodo, por PID	<code>Pid ! Mensaje</code>
Mismo nodo, por nombre registrado	<code>Nombre ! Mensaje</code>
Otro nodo (misma máquina o remota)	<code>{Nombre, Nodo} ! Mensaje</code>

let it crash

sin memoria compartida

solo mensajes

recursión, no bucles

inmutabilidad

Basado en la charla «*Erlang: perdiendo el miedo a la programación concurrente*» de Manuel Montenegro (Facultad de Informática, ciclo Hora del Código). Edición, verificación técnica y actualización de esta hoja: **MC. René Solís R.** Bibliografía: J. Armstrong, *Programming Erlang* (Pragmatic Bookshelf, 2013); F. Cesarini & S. Thompson, *Erlang Programming* (O'Reilly, 2009); F. Hébert, *Learn You Some Erlang for Great Good!* — learnyousomeerlang.com. Todos los fragmentos de código de este documento fueron compilados y ejecutados en Erlang/OTP 29.0.5 (la versión estable descargable en erlang.org al momento de esta edición) para confirmar que la sintaxis y las salidas mostradas siguen siendo válidas.